Lab 4 – VibeCoding

Rock Paper Scissors – Refined Step-by-Step Implementation Guide

Repository: SETAPESU26/59_rock_paper_scissor

Assignment type: Individual

Purpose: Fix the broken game, add the required features using Vibe Coding/LLM prompts, test each change, maintain separate commits, and push the completed work to the student's own fork.

1. Lab Requirements

• Open the assigned repository and read its README.

• Clone or fork the repository.

• Run the original Python game.

• Record a 10-second video before making changes.

• Use Vibe Coding/LLM prompts to fix the broken code.

• Add the README-listed features one by one.

• Record another 10-second video after all changes.

• Push the completed code to your own repository.

• Use a separate Git commit for each task.

• Do not create a Pull Request to the SETAPESU26 main repository.

• Submit the before/after videos, updated code, and exported AI chat history under the Lab-4 folder.

2. Initial Setup

Step 1 – Fork the repository

Open the assigned GitHub repository and click **Fork** → **Create fork**. Work only in your own fork.

Step 2 – Clone the fork

```
git clone https://github.com/YOUR_USERNAME/59_rock_paper_scissor.git
cd 59_rock_paper_scissor
code .
```

Step 3 – Install Pygame

```
python -m pip install pygame
```

Step 4 – Run the original game

```
python main.py
```

Step 5 – Record the BEFORE video

Before changing any code, record about 10 seconds of the original game. Demonstrate the existing inverted-winner bug if possible. Save it as **before.mp4**.

3. Task 1 – Fix the Inverted Winner Bug

File: **game/game_engine.py**

The winning combinations must award the point to the player when the player's move beats the CPU's move.

Replace the outcome rules with:

```
rules = {
    ("ROCK", "SCISSORS"): "PLAYER",
    ("SCISSORS", "PAPER"): "PLAYER",
    ("PAPER", "ROCK"): "PLAYER",

    ("SCISSORS", "ROCK"): "CPU",
    ("PAPER", "SCISSORS"): "CPU",
    ("ROCK", "PAPER"): "CPU",
}
```

Test: ROCK vs SCISSORS → Player wins; SCISSORS vs PAPER → Player wins; PAPER vs ROCK → Player wins; identical choices → Draw.

Commit:

```
git add .
git commit -m "Task 1: Fix inverted outcome evaluation"
```

4. Task 2 – First-to-X / First-to-Five Match

For the final submission, use **5** as the target because the chosen task/commit wording is “First to Five Match Victory.” You may temporarily use 3 while testing, then restore it to 5.

4.1 Add match variables inside __init__()

```
self.target_score = 5
self.game_over = False
self.winner = None
```

4.2 Replace/update play_round()

```
def play_round(self, choice):
    if self.game_over:
        return

    self.player_choice = choice
    self.cpu_choice = random.choice(self.choices)

    outcome = self.determine_winner(
        self.player_choice,
        self.cpu_choice
    )

    if outcome == "PLAYER":
        self.player_score += 1
        self.result_text = (
            f"You Win! {self.player_choice} beats {self.cpu_choice}."
        )
        self.result_color = (80, 230, 120)

    elif outcome == "CPU":
        self.cpu_score += 1
        self.result_text = (
            f"You Lose! {self.cpu_choice} beats {self.player_choice}."
        )
        self.result_color = (240, 80, 80)

    else:
        self.result_text = (
            f"It's a Draw! Both picked {self.player_choice}."
        )
        self.result_color = (240, 210, 80)

    self.showing_result = True
    self.round_resolved_time = pygame.time.get_ticks()

    if self.player_score >= self.target_score:
        self.game_over = True
        self.winner = "PLAYER"
        self.result_text = "PLAYER WINS THE MATCH!"
        self.result_color = (80, 230, 120)

    elif self.cpu_score >= self.target_score:
        self.game_over = True
        self.winner = "CPU"
        self.result_text = "CPU WINS THE MATCH!"
        self.result_color = (240, 80, 80)
```

4.3 Add reset_game()

```
def reset_game(self):
    self.player_choice = None
    self.cpu_choice = None
    self.player_score = 0
    self.cpu_score = 0
    self.result_text = "Make your move!"
    self.result_color = (220, 225, 235)
    self.showing_result = False
    self.game_over = False
    self.winner = None
```

4.4 Handle R to restart

```
def handle_event(self, event):
    if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
        if self.game_over:
            self.reset_game()
            return

    if self.game_over:
        return

    # Keep the repository's existing mouse/button handling below
    # this point.
```

4.5 Game-over display

Inside render(), after drawing the normal game elements and buttons, display a dark transparent overlay when self.game_over is True, followed by the winner and “Press R to restart”.

```
if self.game_over:
    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA
    )
    overlay.fill((0, 0, 0, 170))
    screen.blit(overlay, (0, 0))

    game_over_font = pygame.font.SysFont(None, 50)

    winner_text = (
        "PLAYER WINS!"
        if self.winner == "PLAYER"
        else "CPU WINS!"
    )

    winner_surf = game_over_font.render(
        winner_text,
        True,
        (255, 255, 255)
    )

    screen.blit(
        winner_surf,
        (
            self.width // 2 - winner_surf.get_width() // 2,
            self.height // 2 - 40
        )
    )

    restart_font = pygame.font.SysFont(None, 28)

    restart_surf = restart_font.render(
        "Press R to restart",
        True,
        (220, 220, 220)
    )

    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            self.height // 2 + 20
        )
    )
```

Test: reach 5 points, verify the game-over screen appears, press R, and verify both scores reset to 0.

```
git add .
git commit -m "Task 2: Add first to five match victory"
```

5. Task 3 – Adaptive CPU Strategy

The CPU should observe the player's recent choices and bias its next move toward the counter to the player's most common choice.

5.1 Add history variables inside __init__()

```
self.player_history = []
self.history_limit = 8
```

5.2 Add the adaptive CPU function

```
def get_adaptive_cpu_choice(self):
    if len(self.player_history) < 3:
        return random.choice(self.choices)

    counts = {
        "ROCK": self.player_history.count("ROCK"),
        "PAPER": self.player_history.count("PAPER"),
        "SCISSORS": self.player_history.count("SCISSORS")
    }

    most_common = max(counts, key=counts.get)

    counter_move = {
        "ROCK": "PAPER",
        "PAPER": "SCISSORS",
        "SCISSORS": "ROCK"
    }

    if random.random() < 0.7:
        return counter_move[most_common]

    return random.choice(self.choices)
```

5.3 Record player history and use adaptive selection

```
self.player_choice = choice

self.player_history.append(choice)

if len(self.player_history) > self.history_limit:
    self.player_history.pop(0)

self.cpu_choice = self.get_adaptive_cpu_choice()
```

5.4 Reset history in reset_game()

```
self.player_history = []
```

Test: repeatedly choose ROCK and observe the CPU increasingly favouring PAPER; repeatedly choose PAPER and observe it favouring SCISSORS.

```
git add .
git commit -m "Task 3: Add adaptive CPU strategy"
```

6. Task 4 – Procedural Icons and Reveal Animation

Draw Rock, Paper and Scissors directly with Pygame rather than relying on external image files. Add a short reveal/shake animation before the final choices are shown.

6.1 Add animation state inside __init__()

```
self.reveal_active = False
self.reveal_start_time = 0
self.reveal_duration = 900
```

6.2 Add draw_gesture_icon()

```
def draw_gesture_icon(self, screen, choice, center, size=55):
    x, y = center

    if choice == "ROCK":
        pygame.draw.circle(
            screen, (150, 150, 160),
            (x, y), size // 2
        )
```

```
            pygame.draw.circle(
                screen, (190, 190, 200),
                (x - 12, y - 10), size // 6
            )

        elif choice == "PAPER":
            rect = pygame.Rect(
                x - size // 2,
                y - size // 2,
                size,
                size + 15
            )
            pygame.draw.rect(
                screen, (230, 230, 240),
                rect, border_radius=6
            )

            for i in range(3):
                pygame.draw.line(
                    screen, (150, 150, 160),
                    (x - 15, y - 10 + i * 10),
                    (x + 15, y - 10 + i * 10),
                    2
                )

        elif choice == "SCISSORS":
            pygame.draw.line(
                screen, (210, 210, 220),
                (x - 25, y - 20),
                (x + 20, y + 20),
                8
            )
            pygame.draw.line(
                screen, (210, 210, 220),
                (x - 25, y + 20),
                (x + 20, y - 20),
                8
            )
            pygame.draw.circle(
                screen, (180, 180, 190),
                (x - 22, y - 20), 9, 3
            )
            pygame.draw.circle(
                screen, (180, 180, 190),
                (x - 22, y + 20), 9, 3
            )
```

6.3 Add draw_reveal_animation()

```
    def draw_reveal_animation(self, screen):
        elapsed = pygame.time.get_ticks() - self.reveal_start_time

        if elapsed >= self.reveal_duration:
            self.reveal_active = False
            return False

        progress = elapsed / self.reveal_duration
        shake = int(10 * (1 - progress))

        if (elapsed // 100) % 2 == 0:
            offset = shake
        else:
            offset = -shake

        font = pygame.font.SysFont(None, 30)

        text = font.render(
            "GET READY...",
            True,
            (255, 255, 255)
        )

        screen.blit(
            text,
            (
                self.width // 2 - text.get_width() // 2,
                100
            )
        )

        self.draw_gesture_icon(
            screen,
            "ROCK",
            (self.width // 4 + offset, 170)
```

```
        )

        self.draw_gesture_icon(
            screen,
            "ROCK",
            (3 * self.width // 4 - offset, 170)
        )

        return True
```

6.4 Connect the animation in render()

At the point where the final choices are normally displayed, use self.reveal_active to show the animation first and the actual player/CPU icons after the animation ends. Keep the existing Task 2 game-over overlay at the end of render().

6.5 Start the animation from play_round()

```
self.reveal_active = True
self.reveal_start_time = pygame.time.get_ticks()
```

Place these after the round's player/CPU choices have been selected. Test the visual behavior before committing.

```
git add .
git commit -m "Task 4: Add procedural gesture icons and reveal animation"
```

7. Testing Checklist

Task	What to verify
Task 1	ROCK beats SCISSORS; SCISSORS beats PAPER; PAPER beats ROCK.
Task 2	First to 5 ends the match; winner screen appears; R resets the match.
Task 3	CPU learns from recent player choices and favours counter-moves.
Task 4	Rock/Paper/Scissors are shown as procedural icons; reveal/shake animation appears before final reveal.

8. Git Commit History

```
git log --oneline
```

Expected task commits:

```
Task 1: Fix inverted outcome evaluation
Task 2: Add first to five match victory
Task 3: Add adaptive CPU strategy
Task 4: Add procedural gesture icons and reveal animation
```

9. Final Video and Submission

After video: Run the completed game and record about 10 seconds showing the improved game. Save as **after.mp4**.

Push:

```
git push origin main
```

Do not create a Pull Request to SETAPESU26.

Suggested Lab-4 folder:

```
Lab-4/
■■■ before.mp4
■■■ after.mp4
■■■ Updated Code/
■■■ ChatGPT_Lab4.pdf
```

The assignment handout requires the before/after videos, updated code, and exported chat history under the Lab-4 folder.

10. Important Implementation Note

The code above consolidates the changes discussed during the conversation into a cleaner guide. If your repository's existing method names or surrounding code differ, preserve the repository's existing structure and insert the relevant blocks into the corresponding methods. Do not replace unrelated code.